

**marp: true theme: uncover backgroundColor: #f0f4f8
color: #1a202c**

<!-- Slide 1: Title Slide -->

```
<style>
h1 {
font-size: 3em;
color: #2c5282;
}
h3 {
color: #4a5568;
}
</style>
```

Tic-Tac-Toe: The Classic Game

A C++ Console Implementation

By Arpit Pundir

<!-- Slide 2: Introduction -->

```
<style>
h2 {
color: #2c5282;
}
</style>
```

Introduction

- **What is it?** A classic two-player (X and O) strategy game played on a 3x3 grid.
- **The Goal:** Be the first to get three of your marks in a row—horizontally, vertically, or diagonally.
- **This Project:** A simple, fun, and fully functional implementation of Tic-Tac-Toe written in C++ that runs in the command-line terminal.

<!-- Slide 3: Features -->

Project Features

- **Interactive Gameplay:** Play directly in your console.
- **Two-Player Mode:** Designed for two human players to compete.
- **Clear & Simple UI:** A clean, text-based representation of the game board makes it easy to see the action.

- **Win & Draw Detection:** The game automatically detects and announces the result—whether a player has won or the game is a draw.
- **Input Validation:** Prevents players from making invalid moves (e.g., choosing a box that is already taken).

<!-- Slide 4: How It Works -->

How It Works: The Game Logic

The game operates on a simple, continuous loop until a result is reached.

1. **Initialize & Display:** An empty 3x3 board is created and displayed.
2. **Player Turn:** The current player is prompted to choose a position (1-9).
3. **Update Board:** The chosen position is marked with the player's symbol (X or O).
4. **Check for Result:** After every move, the game checks for:
 - A **Win** (three in a row, column, or diagonal).
 - A **Draw** (all boxes are filled, and no one has won).
5. **Switch Player:** If the game is not over, turns switch to the next player.

<!-- Slide 5: C++ Implementation -->

C++ Implementation Details

- **Language:** C++ was chosen for its performance and control.
- **Core Data Structure:** A simple 2D character array is used to represent the game board.

// Represents the 3x3 grid

```
char board[3][3] = {
    {'1', '2', '3'},
    {'4', '5', '6'},
    {'7', '8', '9'}
};
```

- **Key Functions:**
 - `display_board()`: Renders the current state of the board in the console.
 - `player_turn()`: Handles input from the current player and updates the board.
 - `game_over()`: Contains the logic to check for all win conditions and a draw.
-

<!-- Slide 6: Game in Action -->

A Glimpse of the Game

The game is entirely text-based, making it lightweight and universally runnable.

Initial Board

1	2	3
4	5	6
7	8	9

Game Won

X	O	X
4	O	6
O	8	X

Player 2 [O] Wins!

<!-- Slide 7: Future Scope -->

Future Scope & Improvements

This project has a solid foundation that can be extended in many ways:

- **AI Opponent:** Implement a computer-controlled opponent using algorithms like Minimax for varying difficulty levels (Easy, Hard).
- **Graphical User Interface (GUI):** Rebuild the game using a graphical library (like SFML, Qt, or SDL) for a more modern and visual experience.
- **Score Tracking:** Add a feature to keep track of wins, losses, and draws for each player across multiple games.
- **Network Multiplayer:** Allow two players to compete against each other over a local network or the internet.

<!-- Slide 8: Conclusion -->

<style>

h2 {

font-size: 2.5em;

}

</style>

Thank You!

Questions?